

CS-UY 6063 In-Class Activity #2 Deliverable

Project Description / Feature List / Technical Stack / API List

Complete the following table of team member names / emails / presence today in class.

Name	Email @nyu	Present in class: Y/N
Abhigam Shah	as23662@nyu.edu	Y
Jayant Katiyar	jk7425@nyu.edu	Y
Hena Patel	hp2991@nyu.edu	Y
Vansh Mehta	vm3004@nyu.edu	Y
Manoj Patil	mpp6960@nyu.edu	Y
Riyansh Atul Sachdev	rs10248@nyu.edu	Y
Akshay Yadav	ay3417@nyu.edu	Y
Aayush Salva	as22710@nyu.edu	Y

(continued next page)

Project Vision & Context

Explain the core vision for your Ruby on Rails application. For each section below, write 2-3 paragraphs addressing the following questions. Do not list specific features, page routes, or user workflows in this section. A detailed feature list will be submitted soon. Focus purely on what the application is, who it is for, and why it needs to exist. This section will be shared in Brightspace with all students in the course sections.

What problem does your application address?

College students frequently need to send each other small amounts of money for things like splitting a food order, paying back a friend, or chipping in for a group gift. Existing P2P apps like Venmo, Zelle, and Cash App aren't built for this context. They require linking real bank accounts, expose personal details to a public network of strangers, and offer nothing that connects to campus life. Meanwhile, NYU students separately deal with dining dollars, library fines, and other disconnected campus payment systems that don't talk to each other or to how students actually move money among themselves.

NYU Pay solves this by creating a closed, verified payment network restricted to NYU students, so peer to peer transfers happen in a safer environment than an open platform. It unifies campus spending by letting the same credit balance used to pay back a friend also be redeemed at the cafeteria or library, and it adds a rewards layer that gives students credits for frequent use, something general apps don't offer. A Pro tier also extends this across NYU's many schools, addressing a real gap since Stern, Tandon, CAS, and others currently have no shared peer payment system of their own.

What is your application's target audience or user base?

The main users are NYU students, both undergrad and grad, who already send each other money informally all the time. Splitting rent, paying someone back for lunch, chipping in for a group order, that kind of everyday transaction. These are also the same students who regularly spend money at NYU's own services like dining halls and libraries, so they benefit from having both sides of their spending connected.

Campus merchants and service points, like cafeterias and libraries, are a secondary but important part of the user base. They're not sending peer to peer payments, but they're what makes the credits system actually useful, since students need somewhere real to redeem them.

Down the line, the Pro tier opens things up to students across NYU's different schools, like Stern, Tandon, and CAS. Right now these schools operate pretty separately, so this tier is for students who want to transact or interact across those boundaries while still keeping the overall system

organized by school. The aim is to make transactions more interactive, secure, seamless and easy to perform.

The overall value proposition (your "elevator pitch"). For more information on the elevator pitch, [see this link from MIT](#).

NYU Pay lets genuine NYU students send each other money as easily as Zelle, but instead of that value just sitting in a generic payment app, it turns everyday transactions into credits students can spend right on campus, at places like the cafeteria and library. It brings peer payments and campus spending together instead of keeping them in two separate systems that don't talk to each other.

For students, that means one trusted app for both paying friends and spending on campus life. For **NYU**, it means keeping that financial activity within the university community instead of losing it to outside apps along with added privacy. As the platform grows, the Pro tier extends this same idea across NYU's different schools, turning it into a connected payment network for the whole university instead of just one campus. The difference is that we are integrating the PayDrop mechanism where you can do payments via airdrop or any similar mechanism wirelessly in this application instead of conventional payments methods like tap to pay, Apple Pay etc. This application has the potential to be r.

For instance, if you are at a coffee shop and want to make a quick payment then you do not need to go to the POS terminal and the device detects the POS terminal and initializes the transaction without having to tap your phone or card which in turn reduces the congestion.

(continued next page)

Technical Stack Breakdown

List the core technologies, tools, and libraries you intend to use to build and deploy your application. If a choice is still tentative, note your top choice.

- Backend & Framework: (e.g., Ruby 3.x, Rails 7.x/8.x)
Ruby 3.x, Rails 7.x — full-stack MVC framework handling backend logic, routing, and views.
- Database: (e.g., PostgreSQL, SQLite, MySQL)
 - PostgreSQL — chosen for ACID compliance and reliable transactional integrity, essential for a ledger-based system handling credit balances and transfers.
 - ActiveRecord as the ORM.
- Frontend / UI Strategy: (e.g., ERB + Tailwind CSS, Hotwire/Turbo/Stimulus, React API-only backend, etc.)

React JS + Figma MCP (for frontend prototyping/UI strategy)
- Authentication / Authorization: (e.g., Devise, OmniAuth, custom has_secure_password, Pundit/CanCanCan)

TBD Later
- Key Gems & Third-Party APIs: (e.g., Active Storage + AWS S3, Stripe API, Sidekiq/Redis, Faraday, etc.)
 1. Stripe API
 2. Sidekiq + Redis
 3. Rubocop,
 4. RSpec,
 5. FactoryBot,
 6. Faker,
 7. BigDecimal

- Additional Services (Cache, Messaging Service, SMS, EMail)
Redis, Action Mailer, Twilio (optional, tentative)

TBD later.

Core Application Features (MVP & Stretch Goals)

Itemize the key functional capabilities of your application. Provide a 1–2 sentence description for each feature to explain *what* it does and *who* uses it. Group your list by implementation priority.

For each feature, provide:

- **Feature Name:** A concise, clear title (e.g., *User Profile Management*).
- **Target Persona / Role:** Who uses this? (e.g., *Guest, Logged-in User, Admin*).
- **Short Description:** 1–2 sentences detailing the action the user takes and the expected result.
- **Priority:** Categorize as either **MVP** (Must-have for launch) or **Stretch Goal** (Nice-to-have).

1. Feature Name: Admin Dashboard

Target Persona: Admin

Description: Admin views all transactions across the platform and can flag or reverse suspicious activity. Result: oversight and fraud-prevention tooling for the platform operator.

Priority: MVP

2. Feature Name: Paydrop-based Payments

Target Persona: Logged-in User

Description: User paydrops another user's device to prefill a transfer/redemption instead of manually searching for them. Result: faster in-person transactions.

Priority : Stretch Goal

3. Feature Name: Low-Balance Notifications

Target Persona: Logged-in User

Description: User receives a push/email notification when their balance drops below a threshold. Result: prompts a top-up before the user runs out of credits.

Priority :MVP

4. Feature Name: Split Payment

Target Persona: Logged-in User

Description: User splits a redemption or transfer cost among multiple other users (e.g., splitting a group order). Result: each participant is charged their share automatically.

Priority : Stretch Goal

APIs & Data Integrations

Outline all internal API endpoints your application will expose. For each API provide:

- **Service / Endpoint Name:** The external service name (e.g., *Stripe Checkout*, *OpenWeather API*) or internal path (e.g., `POST /api/v1/orders`).
 - **Short Description:** 1–2 sentences explaining what data is being sent or received and why your app needs it.
-
1. **Stripe Checkout / Payment Intents API** — External. Handles the one-way real-money-to-credits conversion; sends the charge amount and receives payment confirmation so the app knows how many credits to grant.
 2. **POST /api/v1/webhooks/Stripe** — Internal. Receives Stripe's asynchronous payment-success/failure events (signature-verified) so credit balances are only updated after payment is actually confirmed, not just initiated.
 3. **POST /api/v1/transfers** — Internal. Accepts sender, receiver, and amount to execute a peer-to-peer credit transfer entirely within the internal ledger; no external service involved.
 4. **GET /api/v1/wallet** — Internal. Returns the authenticated user's current credit balance for display on the dashboard.
 5. **GET /api/v1/transactions** — Internal. Returns a paginated history of a user's transfers, top-ups, and redemptions for the wallet view.
 6. **POST /api/v1/redemptions** — Internal. Records a credit deduction against a merchant account when a user spends credits at a participating location.
 7. **Devise session endpoints (/users/sign_in, /users/sign_up)** — Internal. Handles authentication so only verified NYU students can access wallet and transfer functionality.